Task 4 - Pandas Data Processing

Task 4.1 - Transaction DataFrame

I denne oppgaven bruker vi Pandas til å organisere transaksjonsdata, konvertere tidspunk til datetime format og sortere transaksjonene.

In [8]:
```
import pandas as pd
```

In [9]:
```
def create_dataframe(transactions: list[dict]) -> pd.DataFrame:
    """
    Gjør en lsite med transaksjoner om til en DataFrame.
    Args: (transactions: list[dict]): liste med transaksjonesdata.
    Returns: pd.DataFrame: transaksjonersortert etter tidspunkt.
    """

    # Lager en DataFrame fra transaksjonslisten
    df = pd.DataFrame(transactions)

    # Gjør tidspunktet om til datetime format
    df["timestamp"] = pd.to_datetime(df["timestamp"])

    # Bruker tidspunket som indeks
    df = df.set_index("timestamp")

    # Storterer transaksjonene kronologisk
    df = df.sort_index()

    return df
```

Task 4.2 - Hourly Revenue

Her bruker vi Pandas til å gruppere transaksjonene etter produktet og beregne summen av prisene for hver time. Vi bruker groupby() og resample("1h") for å gjøre dette.

In [10]:
```
def hourly_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """
    Bergenger summen av transaksjonspriser per time og produkt.
    Args: (df: pd.DataFrame) : DataFrame med transaksjoner.
    Returns: pd.DataFrame :  Summert pris per time og produkt.
    """

    # Grupperer etter produkt og summerer prisene per time
    hourly = df.groupby("item").resample("1h")["price"].sum()

    # Gjør resultatet im til n DataFrame
    return hourly.to_frame(name="price")
```

Task 4.3 - Pivot Table

In [11]:
```python
def transaction_pivot(df: pd.DataFrame) -> pd.DataFrame:
    """
    Create a pivot table showing total transaction revenue per item and status.
    Returns a DataFrame with items as rows and transaction statuses as columns.
    """

    # Group transactions by item and status, summing the prices
    pivot = pd.pivot_table(
        df,
        index="item",
        columns="status",
        values="price",
        aggfunc="sum",
        fill_value=0
    )

    return pivot
```

This function creates a pivot table using Pandas to summarize transaction revenue by item and status. It calculates the total price for each combination of item and transaction status (SUCCESS, REFUND, CANCELLED). Missing combinations are filled with zero.

Task 4.4 - Refund ratio per item

In [12]:
```python
import numpy as np
import pandas as pd

def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> n
    epsilon = 1e-6
    ratio = refund_count / (sales_count + epsilon)
    return ratio
```

In [13]:
```python
def refund_ratio_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """
    Calculate the refund ratio per item using the function from Task 3.2.
    Returns a DataFrame containing the refund ratio for each item.
    """

    # Count transactions for each item and status
    counts = pd.crosstab(df["item"], df["status"])

    # Get the number of successful sales and refunds per item
    sales_count = counts.get("SUCCESS", pd.Series(0, index=counts.index))
    refund_count = counts.get("REFUND", pd.Series(0, index=counts.index))

    # Calculate refund ratios using the function from Task 3.2
    ratios = compute_refund_ratio(
        sales_count.to_numpy(),
        refund_count.to_numpy()
    )

    # Return a DataFrame with item names and refund ratios
    return pd.DataFrame({"refund_ratio": ratios}, index=counts.index)
```

In [ ]:
```python
# Test data for Task 4.3 and 4.4
```

```
test_df = pd.DataFrame({
    "item": ["coffee", "coffee", "tea", "tea", "coffee"],
    "status": ["SUCCESS", "REFUND", "SUCCESS", "CANCELLED", "SUCCESS"],
    "price": [30.0, 30.0, 25.0, 25.0, 30.0]
})

# Call the function and print the result
result = refund_ratio_per_item(test_df)
print(result)

display(transaction_pivot(test_df))
```

```
       refund_ratio
item
coffee          0.5
tea             0.0
```

status	CANCELLED	REFUND	SUCCESS
item			
coffee	0.0	30.0	60.0
tea	25.0	0.0	25.0

Task 4.4 – Results and Discussion

The function calculates the refund ratio for each item by comparing the number of refunds with successful transactions. In the test data, coffee has two successful sales and one refund, giving a refund ratio of approximately 0.5. Tea has one successful sale and no refunds, resulting in a ratio of 0. This demonstrates how Pandas and NumPy can be combined to calculate refund ratios for different products.